

How SonarQube-identified technical debt is prioritized: An exploratory case study

Reem Alfayez^{a,*}, Robert Winn^b, Wesam Alwehaibi^b, Elaine Venson^b, Barry Boehm^b

^a College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia

^b Center for Systems and Software Engineering, University of Southern California, Los Angeles, CA, USA

ARTICLE INFO

Keywords:

Technical debt

SonarQube

Technical debt prioritization

Software

Case study

ABSTRACT

Context: Repaying all technical debt (TD) in a system may be unviable, as there is typically a shortage of resources allocated for TD repayment activities. Therefore, TD prioritization is essential to best allocate such limited resources. Fortunately, one can utilize a static code analysis tool, such as SonarQube, to aid in expediting the TD prioritization process.

Objective: Given that SonarQube is one of the most utilized tools in the context of TD, this exploratory case study seeks to explore how SonarQube-identified TD items are perceived and prioritized for repayment.

Methods: The study was designed, replicated, and conducted in four companies and a master's level course, with a total of 89 participants. The participants were requested to select TD items to include for repayment under a resources constraint.

Results: The results revealed that the overwhelming majority of participants prioritized TD by factoring in a TD item's value and cost, a smaller number prioritized higher value TD items, and only one participant prioritized lower cost TD items. Furthermore, it was revealed that the value of a TD item is subjective and context-dependent, and the majority of participants perceive the cost estimations provided by SonarQube for repaying TD items to be reliable and trustworthy when prioritizing TD.

Conclusion: Based on the results, one can conclude that there is no silver bullet TD prioritization approach that addresses all of a developer's objectives and needs. New TD prioritization approaches should be designed without concentrating on a specific prioritization perspective and should be independent of value estimation methods.

1. Introduction

Technical debt (TD) is a metaphor coined by agile software pioneer Ward Cunningham to account for the added software system effort or cost resulting from taking early software system project shortcuts [1]. The metaphor reflects that debt accumulates interest: the later it is paid, the more it costs [1]. While TD can be treated as an investment to enable rapid delivery in time-critical situations, such as meeting market windows for early adopters, it should be carefully managed to ensure that its negative consequences do not exceed its benefits [1,2]. Specifically, the accumulation of TD in a given system can lead to a significant drop in system quality, increased maintenance costs, eventual system decay, customer dissatisfaction, and a negative impact on software practitioners [3,4].

One of the dangers of TD lies within the fact of how common it is in modern society software systems. Researchers at CAST software found that, out of a cumulative total of 550 million lines of code (LOC), each LOC, on average, had a TD amount of \$3.61 [5]. It can

be difficult at times to recognize the total amount present in a system and the associating costs of TD, as it typically does not have a direct impact on the external behavior of a software system [6]. As such, it is imperative to not delay the management of TD, as it can result in an inflexible and uncontrollable system [4,7,8]. Unfortunately, the negative consequences of TD extend past the system itself and on to the software practitioners as well. InsignTD, a global industrial surveys project on the causes and effects of TD, revealed that TD can result in team demotivation, high team turnover, mounting pressures, decreased productivity, and undesirable, though necessary, activities [4]. Studies like the ones included above illustrate the dangers of accumulating TD and why managing TD is a difficult, though necessary, endeavor.

The abundance of TD adds to the challenge of omitting all existing TD in a given system, though it should be noted that repaying all TD is typically deemed infeasible. This infeasibility can be attributed to the typical resources constraints present within TD repayment activities

* Corresponding author.

E-mail address: reemalfayez@ksu.edu.sa (R. Alfayez).

<https://doi.org/10.1016/j.infsof.2023.107147>

Received 11 May 2022; Received in revised form 5 December 2022; Accepted 3 January 2023

Available online 6 January 2023

0950-5849/© 2023 Elsevier B.V. All rights reserved.

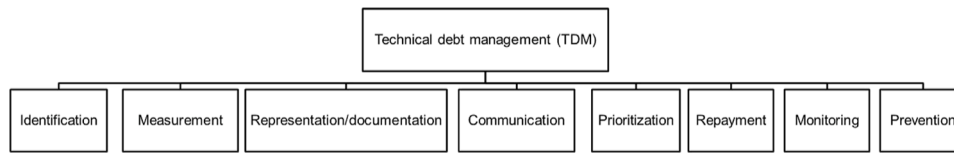


Fig. 1. Technical Debt Management (TDM) activities.

that arise from the innate nature of TD, as repaying TD does not necessarily have a direct, visible impact on the external behavior of a software system [6,9–12]. As a result, both software and business practitioners prefer to spend available resources on developing new features or fixing bugs [6,11,12]. This suggests that there will always be limited resources and tight constraints in place when repaying TD, which forces practitioners to prioritize their TD repayments [9–11]. Moreover, repaying TD is occasionally considered a waste of resources, as repaying some TD items may have little to no benefits [3,13,14].

TD prioritization is the process of deciding which TD items are to be repaid first and which items are to be delayed until later releases [15]. The process has been found to be challenging for not only inexperienced individuals, but even well-motivated and experienced ones [16,17]. Challenges of TD prioritization stem primarily from the limited time and budget allocated for TD repayment activities [9–11,17]. Nonetheless, many TD prioritization approaches have been developed to facilitate the prioritization process. TD prioritization has caught the attention of the software engineering community in recent years, as evidenced by the increasing number of systematic literature reviews that have aimed to analyze the current status of the currently researched TD prioritization approaches [14,18,19]. Previous research on TD has revealed a few research gaps regarding the existing TD prioritization approaches. Specifically, the aforementioned reviews have raised the following concerns: the practicality of approaches in real-world settings, given that many of the identified TD prioritization approaches have not been evaluated in an industry setting; current research may not cover all practical aspects for TD prioritization in real world projects; and there is a lack of consensus on what variables the currently researched approaches are based on and how said variables are measured [14,18,19].

Aiming to contribute to the existing body of TD literature and the body of TD prioritization knowledge, this paper presents an exploratory case study that aims to understand how TD is prioritized [20]. Particularly, the focus of this exploratory case study is the prioritization of TD items identified by SonarQube [21]. SonarQube is a static analysis tool that identifies and measures code TD [22], which refers to code that is poorly written and violates coding rules or best coding practices (e.g., duplicated code) [15]. The decision to focus on SonarQube-identified TD items is derived from the observations that code TD was found to be the most studied and discussed type of TD [15] and that SonarQube is the most referenced tool in the context of TD [23,24]. Therefore, this study primarily focuses on code TD identified by SonarQube. The study was conducted in four companies and a master's level course, and it includes a total of 89 participants: 11 software practitioners and 78 students. Participants of the study were requested to select SonarQube-identified TD items to include for repayment under a resources constraint and fill out a questionnaire to detail their approaches. Subsequently, the results were analyzed to identify the approaches followed to prioritize SonarQube-identified TD items, how the values of these items are perceived, and how their costs are perceived.

The rest of the paper is organized as follows: Section 2 provides background information. Section 3 provides an overview on related studies. Section 4 presents the design of this exploratory case study. Section 5 details the study's results. Section 6 discusses the results, and Section 7 addresses potential threats to the study's validity. Lastly, Section 8 concludes the study.

2. Background

This section provides background information on Technical Debt Management (TDM) and the static code analysis tool SonarQube.

2.1. Technical Debt Management (TDM)

TDM is the process of acknowledging, identifying, measuring, and reducing TD. It is essential to a software system's evolution [25,26], and it should be conducted collaboratively between the technical and business teams to ensure the addressing and balancing of both of their goals [25]. The mapping study conducted by Li et al. [15] concluded that TDM can be described in eight distinctive activities, which are displayed in Fig. 1. The activities are defined based on Li et al. [15]'s definitions as follows: **Identification**: Detecting TD resulting from unintentional or intentional technical decisions in a software system using specific techniques, such as static code analysis; **Measurement**: Estimating the level of the overall amount of TD in a given system or measuring the cost and benefits of the identified TD in a given software system using estimation methods; **Representation**: Uniformly documenting TD while addressing the concerns of all stakeholders; **Communication**: Sharing all identified TD and its effects among all stakeholders to facilitate its discussion and management; **Prioritization**: Deciding which TD items are to be repaid in a given repayment activity and which TD items should be delayed until further releases, based on specific, predefined rules to support the said decision; **Repayment**: Resolving or mitigating identified TD in a given software system using specific techniques, such as refactoring; **Monitoring**: Observing the changes in the value and cost of unpaid TD over time; **Prevention**: Preventing the incurring of potential and future TD.

2.2. SonarQube

SonarQube is a static code analysis tool that can be utilized to identify and measure TD. It was selected for use in this study due to it being, to the best of the authors' knowledge, the only open-source software (OSS) tool that identifies and quantifies TD while also being widely used in industry and OSS communities alike [22,27]. According to SonarQube's website, the tool is used by more than 295,000 organizations around the globe, including the United States Department of Homeland Security, Samsung, and MasterCard. Moreover, the tool was discovered to be used in over 15,000 OSS projects [28]. SonarQube was also found to be the most discussed tool in the context of TD on both StackOverflow and LinkedIn [23,24]. The TD measures of the tool were also found to be the most utilized measures in academic research empirical evaluations [23].

SonarQube aids developers continuously improve their source code by providing measures on TD, security, reliability, and much more. The tool provides a wide range of code analyzers that support over 29 programming languages. Users can integrate the provided code analyzers into various development environments or run the SonarQube analyzer, which is a Java-based command-line tool, to analyze their code. The analysis results are reported in a dashboard format that can be further customized to the user's preferences.

SonarQube utilizes several built-in rules, which are publicly available in [29], to identify and measure TD. In the case that the built-in rules are found to be lacking, users can extend the tool by adding their

own rules [22]. Each rule assigns a detected TD item, which is called a code smell in the tool for each individual issue, a severity score. The severity score of a given TD item is based on the SonarQube team's perception of the said item's probable impact on future maintenance activities. A TD item's severity score can be equated to one of the following from lowest to highest: info, minor, major, critical, or blocker. Rules also assign a remediation effort value, which is the required effort, in minutes, for repayment, to each TD item. The remediation effort may be constant for all identified TD items for some rules, while for others the remediation effort is calculated dynamically based on the specification of each TD item incident [30]. Specifically, the remediation effort of every TD item is totaled to compute the total TD in a system. It is important to note that while SonarQube also identifies security (i.e., vulnerability) and reliability (i.e., bug) issues, these issues are not included in the total TD calculation, as TD is only measured by summing the required effort to fix all code smells according to SonarQube's documentation.

3. Related work

This section summarizes a few related studies and their respective findings. The pool of included studies was limited to ones that empirically evaluated either the usage of SonarQube or TD prioritization with an included human factor. It should be noted that the included summary of related works does not include any study that proposes a TD prioritization approach, as there is a multitude of studies that propose TD prioritization approaches that are not directly related to this study. However, said studies have been analyzed and summarized by a number of previous systematic reviews that are recommended for the reader to review and reference for a more comprehensive summary of works that are focused specifically on TD prioritization, which are [14,18,19,31].

Guo et al. [32] tracked a single TD item in a real software project. The study investigated the reasons behind the decisions to delay and repay the said TD item, in addition to examining the impact of delaying the repayment through Cost-benefit Analysis (CBA). The study found that the repayment of the TD item was delayed due to rushing the release of the project, lack of an immediate impact on current functionality, and uncertainty of the need to implement the change. Though the system's performance was unsatisfactory, the decision for repaying the TD item was delayed until it was deemed mandatory to be repaid. The study simulated the repayment of the TD item in multiple points of time and conducted a CBA at each examined point to show that delaying the repayment of the TD item resulted in increasing its cost by two folds. The study concluded that explicit TD measurement and management can aid in reducing the cost and management of TD.

Saanimaki et al. [33] evaluated the accuracy of SonarQube's remediation time of all three of its issue types (i.e., bug, vulnerability, and code smells). The authors conducted a case study in which 65 third-year-undergraduate students were separated into groups and tasked to fix the identified SonarQube issues in one of 15 OSS projects. The study compared the students' issue's actual fixing time and the tool's estimated remediation time. It revealed that SonarQube's estimated issues remediation times are typically overestimated. Moreover, the study found that the estimated remediation times for fixing bugs are the least accurate estimates, while the estimated remediation times for fixing code smells are the most accurate. The authors extended their research in [34], where they included 16 more students in their pool of participants and six more OSS projects, and confirmed the results of their prior study; SonarQube's estimated remediation efforts may be overestimated.

Tan et al. [35] conducted an empirical study on self-fixed TD. The authors analyzed over 17,000 commits from 20 different Python Apache projects using SonarQube to detect SonarQube rule violations. The study found that over two thirds of the identified TD items were

self-fixed. Furthermore, the self-fixing rate of TD had a negative correlation with the number of commits, number of developers, and size of a project. The study also revealed that self-fixed TD survival time was shorter than that of non-self-fixed TD and TD items that are related to defects tend to be self-fixed and have a short survival time. Tan et al. extended their initial study in [36] by including 16 additional Java Apache projects and performing a more rigorous statistical analysis. Besides confirming the preceding study's findings, the study revealed that developers who contribute more to a project may increase the self-fixing rate and that the longer a developer is involved in a project the less likely they are to fix TD that they personally created.

Lenarduzzi et al. [37] conducted a case study on how 181 junior developers (133 master's level students and 52 fourth-year bachelor's level students at two different universities) resolve SonarQube issues. Students were grouped into separate teams to develop a software project, analyze the said project using SonarQube, and tasked to resolve any identified issues. The results of the study revealed that junior developers tend to incur more code smells than any other SonarQube issue types, resolve more code smells than any other issue types, and that the actual remediation time of issues was a fraction of the estimated remediation time.

Codabux and Williams [3] interviewed 28 industry engineers to understand their perceptions on agile software adoption and how TD affects the development process. The interviews demonstrated that test TD, code TD, and defect TD are the most encountered types of TD, and that architectural TD is the most difficult TD type to repay. The participants indicated that customer requests are the primary driver for TD prioritization, and a smaller number of participants believe that TD severity can aid in TD prioritization.

Bomfim Jr. and Santos [12] interviewed six technical leaders from four software companies to understand how agile teams deal with TD. The study found that decisions relating to the repayment of TD items are based on the impact of a TD item, cost of repaying a TD item, and the opinions of product owners or customers. The study highlighted the difficulty in allocating resources for TD repayment, due to TD repayment's lack of a clear, tangible value.

Martini et al. [38] conducted multiple case studies on architectural TD. The researchers utilized Arcan [39] to analyze four software systems to detect their respective architectural smells and selected a sample of 22 smells. The researchers interviewed nine practitioners, asked them to identify the architectural issues related to the selected smells, and, lastly, fill out a questionnaire. The results revealed a strong correlation between a smell's negative impact and its priority. Furthermore, the results also found a medium-strong positive correlation between cost and priority, as practitioners prefer to repay higher cost items first.

Martini and Bosch [40] investigated information needed by agile product owners and architects to prioritize architectural TD. Participants were found to consider the following aspects, sorted based on the average of importance and frequency: competitive advantage, customer value, market attractiveness, lead time, maintenance cost, customer long term satisfaction, risk penalty, and volatility.

4. Case study design

This section presents the design of this exploratory case study. The design and reporting of the study are based on the guidelines presented by Runeson et al. [20]. The goal of this case study is to primarily explore how SonarQube-identified TD items are prioritized.

4.1. Research questions

To gain a better understanding of how SonarQube-identified TD items are prioritized, this study addresses three specific research questions that act as the driving factors behind the study's design and aid in aligning the study's goals and the researchers' interests.

Prioritization Approaches: Previous research on TD prioritization has illustrated a lack of uniformity on what variables the currently researched TD prioritization approaches are based on and how these variables are specifically utilized during prioritization [14,18,19,31]. Ribeiro et al. [31] have identified and extensively discussed variables utilized during TD repayment, and they refer to them as decision factors. Alfayez et al. [18] compiled the decision factors presented in [31] into a set of three overarching categories, which are referred to as “decision factor categories”: value, cost, and resources constraint. Moreover, the review highlighted that a TD prioritization approach may revolve around one of the categories listed above or a combination of them. However, as the current TD literature indicates that TD repayment is almost always a highly constrained endeavor, with the aim of repaying all TD present in a given system typically being unrealistic [9–11], this study assumes that there will always be a resource constraint that needs to be satisfied when prioritizing TD.

This study's primary objective is to explore how TD items identified by SonarQube are prioritized for repayment. Having such an understanding is particularly important given the overwhelming number of TD items that are identified by static analysis tools in industry software [41] and the popularity of SonarQube, specifically in the industry and OSS projects [22,27,28]. As stated above, TD prioritization revolves around several decision factor categories, most notably being the value and cost of TD items. Given the significance of TD prioritization and the widespread use of SonarQube [22–24,27,28], it may prove beneficial to gain a better understanding on how the value and cost of SonarQube-identified TD items impact one's TD prioritization process. Consequently, this study seeks to understand how SonarQube-identified TD items are prioritized and whether their value, cost, or a combination of the two are accounted for during TD prioritization. This leads the study's first research question to being: **RQ1: How are SonarQube-identified TD items prioritized in terms of their associated value and cost?**

Perceiving Value: Many studies that distinguish successful software projects from failed ones found that the majority of software project failures are caused by value-oriented shortfalls [9,42]. Specifically, the value-based software engineering (VBSE) has divulged that value-neutral approaches to software engineering can be the source of failure for a software project [9], as such approaches can cause software projects to expend significant amounts of scarce resources on activities with negative returns on investment [9,42]. Furthermore, the VBSE argues that value should be integrated into the full range of existing and emerging software engineering principles and practices [9].

The value of a TD item is typically determined by software engineers. It can be estimated based on a simple or complex notion of value and with respect to shared goals or to satisfy personal objectives [9,14,18,19]. When using the term value, this study refers to an ultimate benefit, which may be subjective to the eye of the beholder and based on multiple characteristics [9]. In the context of TD, TD items can be viewed as value-neutral, where each TD item is assumed to be equally important, or can be assigned a value, where the benefit that would be achieved from repaying a given TD item is realized. Fortunately, the software engineering community has developed a wide range of methods to estimate a TD item's value [14,18,19]. However, many of the researched value estimation methods have not been evaluated in an industry setting [15,18,31]. Consequently, whether these estimation methods are followed and the specific circumstances in which these estimation methods are utilized is unknown.

This study revolves around the use of SonarQube and a few of its built-in features, one of which being the assigning of a severity score to TD items. Given that SonarQube assigns a severity score to each TD item based on the item's potential impact and likelihood of said impact to occur, one would assume that a TD item's severity score would contribute to the overall value of a TD item. However, it is not clear whether users are using the provided severity score, not using severity score, or using other factors when determining the value of

TD. Furthermore, there has been no previous study conducted, to the best of the authors' knowledge, that investigates how users perceive the value of SonarQube-identified TD items. Understanding how the value of SonarQube-identified TD items are perceived could prove beneficial, as it may clarify the basis in which value is assigned and whether SonarQube TD items are considered value-neutral or if they vary in terms of their values. Having such an understanding can aid future researchers and practitioners alike in determining how to better estimate and understand the value of SonarQube-identified TD items. Therefore, the second research question is: **RQ2: How are SonarQube-identified TD items' values perceived?**

Perceiving Cost: Both software developers and researchers make use of static code analysis tools to aid in conducting TD-related activities [23,24]. Typically, static code analysis tools that identify and quantify TD provide estimates regarding the cost associated with repaying each TD item [21,41,43]. However, previous studies have noted that costs associated with TD items may be context dependent [10,15,18,31], which refers to TD having varying associated costs according to the perspective at hand. Additionally, software developers, who make use of static code analysis tools, can have varying opinions on the estimated costs of TD that may affect the way software developers utilize estimated costs of TD items. Software developers may deem a tools' estimations as fitting and appropriate for the prioritization process and fully depend on them or they may judge them as unfitting and take steps to modify such estimates based on their previous experiences. In the context of SonarQube, several previous studies have found that the remediation time estimated by SonarQube is inaccurate and, as compared to the actual time spent to fix TD items, is overestimated in most cases [33,34,37]. Therefore, it may prove beneficial to gain more insight into how the provided costs of TD items from SonarQube are perceived. The observations mentioned above lead into this study's third research question being: **RQ3: How are SonarQube-identified TD items' costs perceived?**

4.2. Subjects

The authors employed a convenience sampling technique to select potential study subjects. Convenience sampling is the most common form of non-probabilistic sampling, in which samples are collected by including samples that are conveniently available to researchers [44]. The technique is commonly applied when there is a hurdle in accumulating subjects or when the population is unknown [44]. Inferences and conclusions drawn from such samples cannot be used in generalizations pertaining to the entire population [44]. Regardless of the limitation pertaining to the generalizability of convenience sampling, convenience sampling has proved its effectiveness in preliminary explorations and observations of various phenomena and behaviors [44]. The following sections describe in more detail the study's participants and the utilized software systems within this study.

4.2.1. Participants

Researchers recruited 89 participants from 5 separate sources and then categorized them into two sets. The first set (i.e., 12.36% of the participants) contains six software practitioners from company C1, three practitioners from company C2, one practitioner from company C3, and one practitioner from company C4. Company C1 is a mid-size software development company that develops and maintains web-based, information technology solutions for the healthcare insurance market. Company C2 is an investment company that has a small-size software engineering team of six employees. Company C3 is a software company that specializes in providing solutions for various government sectors to digitize and automate their operations. Company C4 is an information and communication technology provider that provides software and hardware services. The second set (i.e., 87.64% of the participants) consists of 78 students from a master's level, computer science university course. The course is an elective course

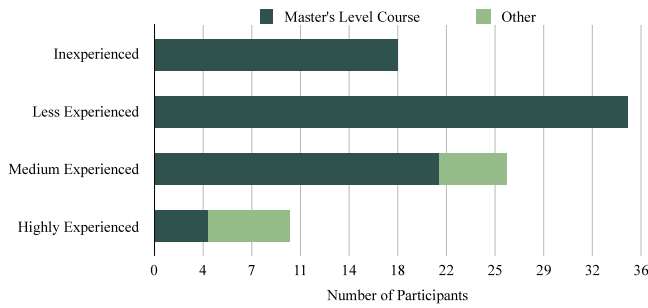


Fig. 2. Breakdown of participants based on industry experience (in years) and affiliation.

that aims to provide students with real-world software engineering experience by assigning groups of them to work on software projects that have real clients, who are university affiliated or external. Client organizations can be identified as the following: for-profit, non-profit, government sector, academic departments, research groups, or any other organization developing a software project. An assigned software project might be a new project, a continuing project from previous semesters, or maintaining an organization's existing project. It should also be noted that the study was advertised to the students in class, and some students volunteered to participate.

Fig. 2 displays the level of industry experience of the study's participants. A stacked column graph was utilized to differentiate between participants, based on their respective affiliations. Dark green indicates participants that are affiliated with the master's level course, and light green indicates participants that are affiliated with one of the participating companies. By following the distribution method denoted by [45], participants were divided into one of four categories based on their previous industry experience at the time this study was conducted: inexperienced (no previous experience), less experienced (less than two years of experience), medium experienced (two to five years of previous experience), or highly experienced (over five years of previous experience). One should note that the naming of each category is exclusive to this study and was not intentionally adopted from any previous study. As Fig. 2 depicts, 18 participants were considered to be inexperienced (i.e., 20.22%), 35 participants were considered to be less experienced (i.e., 39.33%), 26 participants were considered to be medium experienced (i.e., 29.21%), and 10 participants were considered to be highly experienced (i.e., 11.24%).

4.2.2. Software systems

This study involved the use of 13 software systems, with one being an industry system, one being an OSS system, and eleven being from a master's level course with real-world clients. Each of the study's participants participated using one of the involved systems' list of SonarQube-identified TD items. Table 1 summarizes the included software systems, where "Sys#" identifies each system. The summary includes the utilized programming language, number of TD items identified, indicated as "TD_count" in the table, time required to repay all the identified TD items in minutes, displayed in column "TD_time", and the system's thousands of lines of code, indicated by column "KLOC".

The first analyzed software system was an industry system (i.e., "Sys1"), which was currently being worked on by practitioners affiliated with company C1. The second analyzed software system, "Sys2", was Apache VXQuery, which is an OSS system that was suggested to researchers at company C2. The system was performed on by practitioners affiliated with companies C2, C3, and C4, as these companies hesitated in sharing their code due to legal and intellectual property concerns. Lastly, all participants that were affiliated with the master's level course participated in the study with one of the eleven different software systems originating from the master's level

Table 1

Summary of subject software systems.

Sys#	Language	TD_count	TD_time	KLOC
Sys1	Java	5,042	46,829	45.98
Sys2	Java	1,743	17,047	42.13
Sys3	Java	102	768	3.90
Sys4	Java	2,438	50,697	28.99
Sys5	PHP	399	4,199	4.94
Sys6	JavaScript	11,105	147,669	379.44
Sys7	JavaScript	5,179	86,942	1257.50
Sys8	JavaScript	355	6,103	67.45
Sys9	JavaScript	1,290	17,859	345.40
Sys10	JavaScript	237	3,251	76.44
Sys11	Python	115	660	3.88
Sys12	JavaScript	26	520	13.93
Sys13	Java	210	1,523	3.60

course, where each said master's student participant specifically utilized the system that they were currently working on in the course. One should note that the high number of involved software systems was to ensure that each participating master's student would be able to comprehensively complete the study.

4.3. The exploratory case study

The exploratory case study consisted of four phases that are summarized below: introductory session, artifact preparation, TD prioritization activity, and questionnaire. All phases of the study were conducted in one sitting with each participant, in person, and the total time elapsed varied among participants. The researchers visited the participating companies and individually met with students in class to conduct the study.

4.3.1. Introductory session

In the first phase, an introductory session was performed that presented the exploratory study's primary concepts for each participant. During the introductory session, the concept of TD and SonarQube were introduced. Cunningham's definition of TD was provided, and the participants were referred to his video explanation [46] and Fowler's illustration of TD [47]. Subsequently, SonarQube was presented to the participants by displaying an OSS system that was analyzed on SonarCloud, which is the online version of SonarQube. SonarQube's dashboard was then displayed to the participants, and the participants were walked through all of the capabilities of SonarQube, with more of a focus on SonarQube's TD analysis functionality. The total amount of TD and the details of an example for each TD severity type were also presented to the participants. The details for each TD severity type example included the following: the file in which a given TD item exists, start line, end line, severity, time to repay in minutes, a message that suggests how to repay a given TD item, and the rule which a given TD item has violated. Lastly, the list of SonarQube rules [29] was presented to the participants.

This introductory session was necessary to ensure a mutual understanding of the basic concepts of the study. It should be noted that all participants had previous knowledge of TD and experience working with SonarQube. However, regardless of any previous knowledge or experience, all participants were required to partake in the introductory session. Additionally, all included companies in this study utilized SonarQube in their development pipelines prior to their employees participating in this study. Prior to this study, all participating students utilized SonarQube to analyze their respective software systems once and were recommended, though not required depending on their client's preference, to continue using the tool.

4.3.2. Artifact preparation

In the artifact preparation phase, all necessary artifacts needed to conduct the study were prepared. To do as such, each included software system was locally analyzed using SonarQube analyzer and the default rule set for each programming language. The results of SonarQube's analysis for each software system were then presented in SonarQube's built-in dashboard. Afterward, the TD items and their details were extracted using SonarQube's API [48] and saved in an excel sheet. The excel sheet contained the list of the identified TD items and their respective details, including rule, message, severity, file, start line, end line, and time to repay. One should note that each participant was only involved with analyzing the system that they were currently working on at the time the study was conducted and was only provided with information relating to the said system.

4.3.3. TD prioritization activity

During the TD prioritization activity phase, participants were provided with an excel sheet, resulting from the previous phase, containing the list of SonarQube-identified TD items and their respective details. Participants were asked, without any mention to the value or cost of TD, to select TD items for a repayment activity and provide their reasonings. No time limit was put in place during the activity, as participants were allowed to work at their own pace. However, a resource constraint was put in place during prioritization to simulate a real world environment. Specifically, the total cost of the selected TD items for repayment should not exceed 360 minutes. Participants were explained to that the output of the activity should be a list of prioritized TD items for repayment.

4.3.4. Questionnaire

Following the prioritization activity, a questionnaire phase was conducted where participants were requested to fill out a questionnaire, in which they provided the following in a text format: the system they worked on, role, and industry experience. Additionally, participants were asked to answer the following questions also in a text format: how they prioritized SonarQube-identified TD items, what cost estimation method a participant utilized if they utilized any, and what value estimation method a participant utilized if they utilized any. Participants were then asked in a radio button format if different files of a system have different levels of importance. Lastly, participants were requested to select the possible reasons behind variations in value in a checkbox format, with "Other" being an available option that provided an opportunity to detail other reasons that were not provided.

4.3.5. Design improvements

The experiment and questionnaire were developed by the authors based on the current TD prioritization literature [14,18,19]. Once the initial designs had been finalized, the experiment and questionnaire were piloted with four members of the target population. All four participants were invited to individually conduct the experiment and, subsequently, fill out the questionnaire in a read-aloud session, in which participants read questions out loud as well as externalize their thinking process. In regards to the questionnaire, the participants suggested merging the demographic and TD prioritization sections into one section, as it would provide the participants more of a comprehensive overview of the questionnaire. Participants also recommended changing a question that revolved around the rationalization of why some files should not be treated as equally important as others from an open-ended question to a checkbox question. The suggestions mentioned above were taken into account in the final revision of the questionnaire.

4.4. Data analysis

This case study resulted in an excel file for each participant that contained the prioritized list of TD items and all selected TD items to be included in the repayment activity, in addition to a questionnaire response. The data analysis process began with a review of the collection of excel files and the questionnaire responses. Three authors independently reviewed each participant's excel file sheet and compared it to the corresponding questionnaire response, with an aim to identify any inconsistencies that may be present between the two. The intention was to exclude any participant's data if inconsistencies were to be found, as participants were not allowed to make any changes to their TD prioritization approach in their respective excel file when the prioritization activity had concluded and the questionnaire phase had begun. The three authors compared their results and found no inconsistencies among the excel files and questionnaire responses.

It should be noted that a few participants referred to value as benefit, importance, or gain in their excel file, which was considered as value in their questionnaire response and the author's analysis. Similarly, in the excel files, a few participants did not explicitly refer to the cost of TD items as cost. Instead, they used time, effort, and minutes to refer to cost in their responses. Despite the various utilized terms used to refer to cost, all instances of cost in this study refers to SonarQube's estimates of the time required, in minutes, to repay a given TD item.

After the excel files and questionnaire responses had been confirmed to be in alignment, the responses were analyzed using both a qualitative and quantitative analysis. First, a qualitative approach was followed to analyze any open-ended questions related to the TD prioritization process. The analysis was conducted through the means of a thematic analysis [49], which involves reading the data extensively and coding the data using various descriptive labels. Codes derived from a thematic analysis can be generated using an inductive approach, deductive approach, or integrative approach. The inductive approach utilizes a code's start list that has been created prior to the analysis. In the deductive approach, the code's list is generated during the analysis. The integrative approach uses a combination of both inductive and deductive approaches. This study applied an integrative approach to code the data. The analysis was initiated by two authors that created the inductive coding list based on the study's research questions. Subsequently, the two authors independently read and coded the responses, and generated a deductive coding list during the mapping. Through multiple joint sessions, the two authors discussed the results until an agreement was reached. To verify and validate the findings, two additional authors were provided with the coding list and asked to map the data. The results were compared and discussed in a joint meeting among these two authors and one of the previous authors. Discrepancies were resolved through collective discussions until a consensus was reached. A quantitative analysis was then conducted on the closed-ended questions of the questionnaire and the results of the qualitative analysis.

5. Results

5.1. RQ1: How are SonarQube-identified TD items prioritized in terms of their associated cost and value?

Three distinct patterns were identified in the prioritization approaches adopted by the study's participants. Fig. 3 depicts a breakdown of the three patterns based on their utilization by participants. The patterns are described below in order of their frequency of utilization from highest to lowest. Additionally, a few unique phenomena that were discovered when reviewing the participants' prioritization processes are detailed below.

Table 2
Summary of value and cost prioritization techniques and their frequency of utilization.

Technique	Highly	Medium	Low	Inexperienced	Total
Averaging	–	5.26%	–	–	1.52%
CBA	14.29%	21.05%	19.23%	21.43%	19.70%
Greedy	42.86%	42.11%	42.31%	50%	43.94%
Sorting	14.29%	10.53%	15.38%	–	10.61%
WSM	28.57%	21.05%	23.08%	28.57%	24.24%

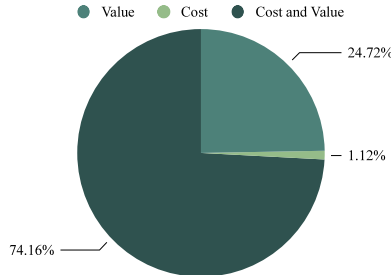


Fig. 3. Breakdown of TD prioritization approaches' patterns utilized by participants.

Value and cost

The majority of the participants (i.e., 66 participants) accounted for the trade-off between value and cost and sought to balance these two factors during their prioritization. 70% of highly experienced participants, 73.08% of medium experienced participants, 74.29% of the less experienced participants, and 77.78% of participants with no previous industry experience accounted for value and cost during prioritization. Participants who accounted for the value and cost of each SonarQube-identified TD item believed that one should take into account the overall quality of a system and distribute the available resources in a way that improves the total obtained value of a given repayment activity. They also believed that one should not spend all of the time designated for a given repayment activity on the highest value TD item if it were to consume all of the allocated time. On the contrary, these participants deduced that repayment activities should aim to optimize the allotted time for repayment and seek to improve the net value of the repayment activity, which can be achieved by considering the economic value of each TD item and being as cost-effective as possible. Moreover, participants who accounted for both the value and cost of TD items also believed that solely depending on the value of TD items is not realistic, as TD repayment activities are nearly always highly constrained in real-world settings. Therefore, individuals should be greedy when spending available resources to maximize the overall value achieved.

A few participants commented that TD prioritization can be thought of as the canonical restricted 0-1 knapsack problem [50]. Both the TD prioritization problem and the knapsack problem have a constraint in place that needs to be satisfied and objectives that are sought to be achieved and balanced. To achieve the goals mentioned above, participants applied multiple techniques, which are listed in Table 2. The table presents the percentage of utilization among the 66 participants who prioritized TD using value and cost and the percentage of utilization among each experience category (i.e., out of the said 66 participants). As the table depicts, greedy was utilized by 43.94%, Weighted Sum Model (WSM) by 24.24%, CBA by 19.70%, sorting by 10.61%, and averaging by 1.52% of the 66 participants.

Value

A smaller number of participants (i.e., 22 participants) prioritized the identified TD items based solely on the value of each TD item. In regards to the experience of value-driven participants, only 30% of highly experienced participants, 26.92% of medium experienced participants, 22.86% of less experienced participants, and 22.22% of inexperienced participants solely accounted for value during their prioritization. Participants that accounted solely for the value of TD

believe that a repayment activity should first tackle the TD items with the highest values and that all available resources should be utilized to address the highest value TD items, regardless of their associated costs. For example, if a TD item were to have the highest value and require 360 minutes to repay, then all of the available 360 minutes should be utilized to repay the said TD item, even if repaying other TD items may provide more overall improvement to the quality of the given system. The rationale of the value-oriented set of participants is that if a TD item is valuable, then it should be repaid first, no matter the cost.

Cost

Only one participant, who was of less experience, prioritized TD items based solely on their repayment cost. The participant calculated the total time required to repay all the TD items in each system's file. Subsequently, the participant sorted the files in an increasing order of the calculated total time required to repay all the TD contained in each file. They then selected files to repay based on the generated sorted list of files. The participant believed that starting with the lowest-cost-to-repay files reduces the overall number of TD items and the number of tasks that the said participant is supposed to complete, which may have a positive impact on their morale and productivity.

Context-switch overhead

A total of four participants considered the context-switch overhead resulting from alternating between different files. The aforementioned set of participants was made up of one medium experienced participant and three less experienced participants. These participants cumulatively prioritized TD items in a file, which means that the participants prioritized each file instead of each TD item. Conversely, the majority of the participants (i.e., 85 participants) considered each individual TD item independently, without taking the context-switch overhead into account.

Security issues or defects as TD items

The questionnaire uncovered that a small group of participants (i.e., 11 participants) deemed a few of the TD items as security issues or defects that needed to be repaid prior to release. These participants do not include any TD items deemed as security issues or defects in their prioritization process, due to the items' innate nature. Due to past experiences, this set of participants realized the consequences and implications of such TD items. As a result, said individuals deemed that these TD items are to be addressed and removed from the system before production, regardless of their associated repayment cost. Examples of such TD items are the return of null from a boolean method and the usage of wildcard imports. This set of participants includes 60% of highly experienced participants, 7.69% of medium experienced participants, 8.57% of less experienced participants, and none of the inexperienced participants.

5.2. RQ2: How are SonarQube-identified TD items' values perceived?

The TD prioritization activity found that there is no consensus on the utilized method to associate a value with each SonarQube-identified TD item within the study's participants. The participants developed multiple valuation formulas, with only one participant not considering value in their TD prioritization process. Table 3 summarizes the valuation formulas created by the participants and their frequency

Table 3
Summary of value formulas and their frequency of utilization by participants.

Value formula	Highly	Medium	Low	Inexperienced	Total
file * severity	50%	50%	50%	55.56%	51.14%
severity	–	15.38%	17.66%	11.11%	13.64%
cost * file * severity	20%	3.85%	2.94%	5.56%	5.68%
cost * severity	10%	11.54%	2.94%	–	5.68%
file * rule	10%	3.85%	5.88%	–	4.55%
severity * rule	10%	3.85%	2.94%	5.56%	4.55%
file * severity * rule	–	3.85%	–	11.11%	3.41%
file + severity	–	3.85%	5.88%	–	3.41%
file	–	–	5.88%	–	2.27%
(x * file) + (y * severity)	–	3.85%	–	5.56%	2.27%
cost * file * rule	–	–	2.94%	–	1.14%
lines * severity * rule	–	–	2.94%	–	1.14%
severity + rule	–	–	–	5.56%	1.14%

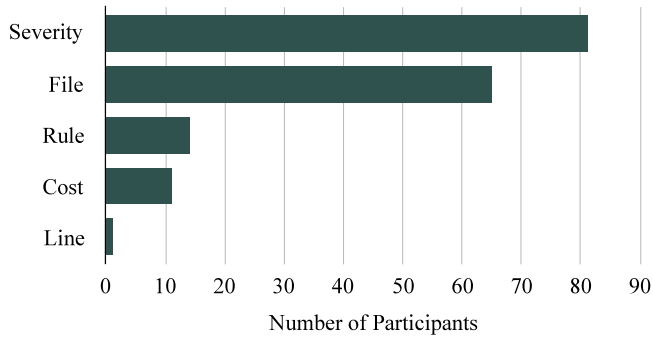


Fig. 4. Breakdown of valuation parameters based on their utilization by participants.

of utilization based on participant's previous industry experience level. Additionally, the "Total" column refers to the total utilization across all participants that considered value during prioritization (i.e., 88 participants). One should note that the "Low" column was calculated out of 34 participants, as one low experienced participant did not consider value during their TD prioritization. The formula "file*severity" was found to be the most employed formula, with 51.14% of the study's participants calculating the value of SonarQube-identified TD items using it. No other value formula comes close in terms of utilization, with "severity", "cost*file*severity", and "cost*severity" following with total utilization values of 13.64%, 5.68%, and 5.68%, respectively.

Several participants (i.e., 14 participants) used one parameter in their valuation, while the majority of participants (i.e., 74 participants) used multiple. When combining the valuation parameters, participants applied multiplication, summation, or WSM functions. The majority of the participants (i.e., 68 participants) used simple multiplication, four of the participants used summation, and only two participants used WSM.

Fig. 4 illustrates the valuation parameters and the frequency of utilization of each parameter. The most utilized parameter was severity, which was followed by file, rule, cost, and number of LOC involved with TD items. The parameters are discussed below in order of their frequency of utilization from highest to lowest.

Severity

As previously mentioned, SonarQube assigns a severity score to each TD item, which are categorized from lowest to highest as follows: info, minor, major, critical, or blocker. When developing a value formula, 92.05% of participants (i.e., 81 of the 88 participants), who accounted for value during their TD prioritization process, considered the severity of TD items. Moreover, 90% of highly experienced participants, 96.15% of medium experienced participants, 85.30% of less experienced participants, and 100% of inexperienced participants utilized severity in their valuation formulas. Participants were found to map severity ranks using one of three distinct methods: linear function, exponential function, or

relative sizing [51]. Each utilized method is explained in more detail below.

- Mapping severity ranks using a linear function involves assigning numeric values to each level of severity, where the values grow by the same amount as the level of severity increases. An example of utilizing a linear function is the following: {info, minor, major, critical, blocker} → {1, 2, 3, 4, 5}. A total of 56 participants employed the use of a linear function to map severity ranks. 90% of highly experienced participants, 65.38% of medium experienced participants, 52.94% of less experienced participants, and 66.67% of participants with no previous experience used a linear function. However, there was no consensus among all participants on the initial numerical value for the lowest severity rank.
- Utilizing an exponential function to map severity ranks is done by selecting a base constant, raising said constant to a linearly increasing power, and then assigning the resulting values to the severity ranks in an increasing order. For example, a participant who utilizes an exponential function could map severity ranks as the following: {info, minor, major, critical, blocker} → {1, 3, 9, 27, 81}. Only 8 participants employed the use of an exponential function, and there was no consensus on the base value of the exponential function. None of the highly experienced participants, 3.85% of medium experienced participants, 14.71% of less experienced participants, and 11.11% of inexperienced participants used an exponential function. It should be noted that participants who utilized an exponential function did so to express greater differences among the severity ranks than that of a linear function. The value assigned to the highest severity rank in a linear function is typically only marginally larger than the one assigned to the lowest rank, while the assigned value to the highest severity rank in an exponential function is significantly larger than that of the lowest rank.
- When following a relative sizing method [51], participants do not utilize a specific mathematical function when mapping severity ranks. On the contrary, they tend to use relative severity mappings. An example of utilizing relative sizing is as follows: {info, minor, major, critical, blocker} → {1, 3, 5, 8, 13}. A total of 17 participants utilized relative sizing when mapping severity ranks. None of the highly experienced participants, 26.92% of medium experienced participants, 17.65% of less experienced participants, and 22.22% of inexperienced participants utilized a relative sizing method. Participants took a relative sizing method due to them having a different perspective of the relative differences among severity ranks. As a result, the resulting mappings were unique to each participant.

Relative importance of a file

System files can have relatively different values of importance that can be utilized when prioritizing TD. However, determining the importance of a file is not a trivial task, as it varies among individuals and

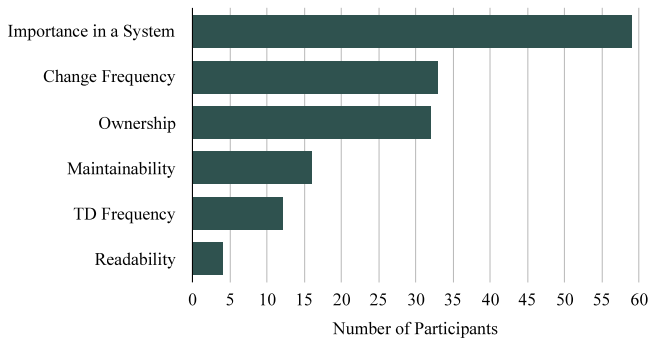


Fig. 5. Breakdown of attributes selected to evaluate files.

the situation at hand [52]. Around 72.73% of the study's participants (i.e., 64 of the 88 participants) that accounted for value during their TD prioritization process incorporated the relative importance of a file in their valuation method. A few participants stated that they did not consider the importance of files when prioritizing TD during the TD prioritization activity due to all files in the system at hand having a relatively similar level of importance. However, they provided the reasons that they would typically consider when determining the relative values of files in other systems, which lack a consistent level of file importance. As such, the measures of whether a given participant believes an attribute is responsible for the importance of a file for each identified attribute are taken in the context of all participants that accounted for value during their TD prioritization (i.e., 88 participants). These participants may have indicated multiple attributes to evaluate a file's importance, as they believe that the importance of a given file depends on the purpose of a repayment activity and varies based on the current circumstances, which indicates that a file's importance is context dependent. Fig. 5 summarizes the attributes that the participants might consider when assigning value to files. The attributes that the participants indicated are explained below based on their frequency of selection.

- File importance in a system: 67.05% of the participants believe that the importance of a file is determined by its value to the system. A file's value to a given system can be based on its business value, customer value, or the set of features that it implements. Furthermore, 70% of highly experienced participants, 69.23% of medium experienced participants, 58.82% of less experienced participants, and 77.78% of inexperienced participants believe that the importance of a file is determined by its value to the system at hand.
- Change frequency: 37.5% of the participants perceive the importance of a file to be its frequency of change, as repaying TD items in a file that is frequently changed may improve the file's maintainability and save future efforts. Furthermore, 70% of highly experienced participants, 46.15% of medium experienced participants, 29.41% of less experienced participants, and 22.22% of inexperienced participants perceive the importance of a file to be its frequency of change.
- 1. Ownership: 36.36% of the participants believe that files that a user owns and frequently interacts with should have higher importance, as they surmise repaying TD items in such files is relatively easier. They also believe that repaying TD items in such files improves the files' maintainability, which may improve the user's productivity. Furthermore, 30% of highly experienced participants, 38.46% of medium experienced participants, 35.29% of less experienced participants, and 38.89% of inexperienced participants believe that files that a user owns and frequently interacts with should have higher importance.

- Maintainability: A few participants (i.e., 18.18%) assign greater importance to files that suffer from maintainability issues. The maintainability of a file can be estimated manually or automatically using software metrics, such as a maintainability index (MI) [53]. Furthermore, 30% of highly experienced participants, 26.92% of medium experienced participants, 17.65% of less experienced participants, and none of the inexperienced participants believe that the importance of a file is based on its maintainability.
- TD frequency: 13.64% of the participants believe that files that contain higher amounts of TD items should have a higher importance. They believe that reducing the amount of TD items present in a file will ease the changeability of the file and improves its maintainability. Furthermore, none of the highly experienced participants, 11.54% of medium experienced participants, 17.65% of less experienced participants, and 16.67% of inexperienced participants believe that files that contain higher amounts of TD items should have a higher importance.
- Readability: A handful of the participants (i.e., 4.55%) consider files that are difficult to read to be more important. A file's readability may be assessed manually or automatically using a variety of metrics, such as the readability metric [54]. Furthermore, 10% of highly experienced participants, 3.85% of medium experienced participants, 2.94% of less experienced participants, and 5.56% of inexperienced participants consider files that are difficult to read to be more important.

The authors' acknowledge that a few of the attributes listed above could be considered to be overlapping, such as maintainability and number of TD items. However, it is believed that each of these attributes can provide a unique insight when assessing software files ease of maintenance. For example, TD has the advantage of identifying specific parts of the software that are in need of maintainability improvement at the LOC level, while other maintainability metrics, such the MI-type metrics, have similar localization and efficiency advantages, but are more at the module level rather than at the LOC level [55].

Rules

Only 14 participants believed that different SonarQube rules have different values. The aforementioned group of participants believed that the SonarQube rule that each TD item violates indicates the impact of the said TD item on a software system. In other words, the violation of some SonarQube rules hold more value than others. Thus, some TD items are more valuable than others in the eyes of a few participants. Only 20% of highly experienced participants, 11.55% of medium experienced participants, 14.70% of less experienced participants, and 22.22% of inexperienced participants utilized rules when evaluating TD.

Cost

A total of 11 participants utilized the cost of TD items in their valuation formula. Only 30% of highly experienced participants, 15.38% of medium experienced participants, 8.82% of less experienced participants, and 5.56% of inexperienced participants utilized cost. These participants perceive cost as a proxy of the importance of each TD item, where a TD item with a higher cost of repayment should have a greater value.

Lines of code (LoC)

Only one participant, who was of less experience, considered the number of LOC involved with TD items. The participant believed that TD items that affect a larger number of LOC should have a higher value.

5.3. RQ3: How are SonarQube-identified TD items' costs perceived?

It was revealed that nearly all of the participants found the provided costs for SonarQube-identified TD items to be trustworthy and not needing to be changed. Only one participant, who was of less experience, changed the cost estimations provided by SonarQube during their TD prioritization approach. The aforementioned participant altered the provided estimates by multiplying them with a scalar. Specifically, they estimated the value of the scalar for each TD item based on their familiarity with the system's component in which the TD item exists and their previous experience with repaying similar TD items to the one at hand. For example, if the participant has worked extensively on a system component in the past, then the TD item associated with the familiar system component is assumed to have less of a cost than that of a similar one in a different system component that the participant is less familiar with. Additionally, a separate, highly experienced participant commented that SonarQube may overestimate the effort required to repay each TD item. However, the participant perceived this as an advantage. They believe that it is always better to overestimate rather than underestimate, which may lead to delays or missed deadlines, and that the tool takes into account the cost of ramification changes, such as rerunning software tests. The participant highlighted that fellow team members, in addition to management, tend to trust the tool's estimates more than the participant's estimates.

6. Discussion

This study's results, most notably the ones deriving from RQ1, illustrate that there is no silver bullet approach that addresses all of the various needs of software developers when prioritizing SonarQube-identified TD items, as there were three distinct prioritization patterns utilized by the study's participants. The three unique prioritization patterns identified in RQ1 can be viewed as the middle ground and opposite ends of a spectrum. The prioritization of SonarQube-identified TD items based on the said items' value or cost are conflicting objectives, with incorporating both being considered a middle-ground approach. The results revealed that the overwhelming majority of participants (i.e., 66 of 89) prioritized the SonarQube-identified TD items by taking into account both the value and cost of the TD items. Moreover, a smaller number of participants (i.e., 22) only took the value of TD into account when prioritizing, and one participant solely took into account the cost of TD. Therefore, given the variety in what to account for in the applied prioritization approaches, it may be unfeasible to identify or develop a single TD prioritization approach that can be applied to all situations, which is in accordance with the findings of several previous studies on TD prioritization [14,18,19].

Additionally, it was difficult to draw a line between experienced participants and those who lack more experience, as the rate of participants that account for the trade off between value and cost during prioritization was relatively similar for all four experience categories. This was also found to be the case among the group of participants that prioritized TD solely based on its value. Furthermore, only one participant who was of less experience prioritized lower cost TD items. Given that the said participant was less experienced and no other participant prioritized using a similar approach, it could be inferred that prioritizing SonarQube-identified TD items based solely on cost may not be the most popular or efficient approach.

One of the revelations of the results, specifically of those from RQ2, was that participants were found to perceive the value of SonarQube-identified TD very differently. In particular, participants employed a wide range of methods when valuing TD items. Though previous research on TD prioritization has revealed that many different methods exist to assign value to TD [14,18,19], the results of this study highlight that even when participants utilize the same static code analysis tool (i.e., SonarQube), participants' perceptions of and methods to calculate a TD item's value can vary substantially. Moreover, not only were

the valuation methods found to vary among participants, but several participants also indicated that they may value a given TD item differently at different points of time depending on the situation at hand based on the results of assigning value to different files, which aligns with previous research on TD that the value of a TD item is context-dependent [14,18,19].

The results revealed that severity score, file, associated cost, LOC, and rule were all factors when assigning value to TD. The SonarQube-assigned severity score of TD items was found to be the most utilized parameter in participant's valuation formulas, which was expected as severity measures the probable impact that a given TD item may have. However, participants were found to have conflicting viewpoints on SonarQube's severity categorization. It was found that they utilized one of three distinct methods when accounting for a TD item's severity score: linear, exponential, or relative sizing. While the majority of participants employed the use of a linear method, a few participants chose to utilize relative sizing or exponential, though no experienced participants utilized either of these two methods. Furthermore, it was revealed that the most utilized combination of parameters to value TD items was severity and file, with the majority of all of the participants across all of the participant experience categories opting to account for these two parameters. The results also unveiled that assigning value to a file is not a trivial task, as participants' responses indicated that a file may be evaluated differently based on the situation at hand and that there is no one method that participants always follow when assigning value to files.

Though it is one of the most important aspects of TD, the cost of TD is a difficult variable to factor in when conducting TD prioritization [14,18,19]. Fortunately, static code analysis tools can aid in expediting the TD prioritization process by providing cost estimates that one can utilize when conducting their prioritization. SonarQube is one such tool that provides users with TD item cost estimates. As the results of RQ3 illustrate, all but one participant fully relied upon the cost estimates for TD provided by SonarQube, which was quite surprising. It was expected that more participants would have altered or discounted the cost estimates due to several previous studies finding that SonarQube may overestimate the required remediation time for TD and could be inaccurate at times [33,34,37].

This study does not aim to support nor undermine any particular TD prioritization approach. On the contrary, it aims to identify the patterns in TD prioritization approaches utilized by software developers when presented information on TD items' value and cost by SonarQube, without deeming an approach as right or wrong. The results of the study illustrate that even when the same static code analysis tool is utilized, software developers may have differing perspectives and opinions on how TD should be prioritized. Adopting any TD prioritization approach depends on the ever-changing needs of software developers and the situation at hand. The results of this study can be used as motivation for the software engineering community to continue supporting the various needs and objectives of software developers through the expansion of the scope of SonarQube and other similar static code analysis tools. For example, a SonarQube plugin can be created using the results of the study. The plugin can provide its users the ability to select how they want the identified TD to be prioritized (e.g., emphasis on value, cost, or combination of them). It can also allow users to select how the prioritization will be conducted, such as CBA, WSM, greedy, and sorting. The plugin can allow users to select one of the valuation formulas included in the study, in addition to selecting their own. The plugin can also automate the calculation of file importance when possible.

7. Threats to validity

This section discusses potential threats to this study's validity and their corresponding mitigation strategies, while following the guidelines of case study research presented in [20].

7.1. Construct validity

The construct validity of a study is concerned with the degree to which the utilized scales, metrics, and instruments actually measure what they are supposed to measure [20]. A potential threat to the construct validity of this study is the accuracy of the measurement of TD. Aiming to mitigate this threat, this study relied on SonarQube to measure TD. To the best of the authors' knowledge, SonarQube is the only OSS tool that identifies and measures TD while also being widely utilized in both the industry and OSS communities [22,27]. Furthermore, all included software systems were analyzed using SonarQube's default set of rules to avoid subjectivity and error, as using SonarQube's default rules ensures that all included systems were assessed with a unified set of rules. Another potential threat to the construct validity of this study is the varying interpretations of value and cost. This threat was mitigated by first asking participants to complete a questionnaire, in which they detailed their rationales behind their usage of value and cost, and then comparing the questionnaire responses with the resultant files of the prioritization process. The comparison step ensured a mutual understanding of value and cost, in addition to uniformity among the questionnaire responses and resultant prioritization files. Lastly, a third potential threat to this study's construct validity is the possibility that participants may have misinterpreted questions in the study's questionnaire phase. To mitigate this threat, the authors conducted a pilot study with a few software practitioners to identify any potential shortfalls and updated the questionnaire to reflect their feedback.

7.2. External validity

The external validity of a case study is concerned with defining a domain to which the obtained findings can be applied, beyond the circumstances of the study itself and to more general situations [20]. Though this exploratory case study was replicated and conducted in four companies and a master's level course, neither the comprehensibility nor the generalizability of the acquired findings can be claimed. This study utilized a convenience sampling technique to recruit its participants. Inferences and conclusions drawn from convenience samples cannot be used in generalizations pertaining to the entire population [44]. The primary goal of this study is to unveil and explore patterns in TD prioritization approaches utilized by software developers when prioritizing SonarQube-identified TD items for repayment, without aiming to draw any generalizable conclusions. As such, the authors acknowledge that there may exist other TD prioritization patterns that this study failed to identify. Additionally, due to difficulty arising in convincing companies to agree on sparing a portion of their software developers' working-hours and revealing their intricacies, the initial limited pool of industry participants was expanded by including students of a master's level course. The decision to include students was made based on several key insights: previous, related studies have relied on bachelor's students [33,34,37]; the usage of students is considered appropriate in software engineering studies [56,57], especially when the aim is to gather initial evidence; and that the majority of the included students had previously worked in the industry. However, one can still argue that some participants are lacking in experience and that the results would differ if the pool of participants included more experienced software developers. The authors acknowledge this limitation, though it is emphasized that the primary goal of the study is to identify prioritization patterns, and that the patterns revealed from software developers that may lack experience are also valuable.

Another threat to the external validity of this study is that the majority of the included systems are Java-based. It should be noted that the authors had little control over the utilized systems. The included systems were the systems that the master's level students had been currently working on, a system that the participants of one of the included companies were working on, and a system that one participant suggested for use, which other participants felt comfortable using. As such, the authors admit this limitation in the study.

7.3. Reliability

A study's reliability is concerned with the dependability of the study and its findings on the researchers who conducted the study [20]. This study's setup and data analysis steps were detailed above to allow for future replication. Though the authors anticipate that other researchers will come to similar conclusions if the same setup, subjects, and data analysis steps were followed, there is still a chance of inconsistencies occurring resulting from reliance on human judgment when analyzing the data. To mitigate any potential introduced biases, the analysis process was conducted in accordance with the empirical standards for software engineering research and following the recommended steps for thematic synthesis in software engineering [49,58]. However, it should be noted that there is no way to fully guarantee that two humans will have consistent analyses, nor entirely avoid any introduced biases resulting from human judgment.

8. Conclusion

This study summarizes a few of the mindsets and behaviors of software developers when prioritizing SonarQube-identified TD items. We conducted an exploratory case study where 89 participants were requested to select SonarQube-identified TD items for repayment and, subsequently, tasked with completing a questionnaire, in which they detailed their utilized prioritization approach and associating rationales. The results revealed that the overwhelming majority of the participants balanced the trade-off between the value and cost of TD items. In contrast, a smaller number of participants prioritized higher value TD items, and only one participant prioritized lower cost TD items. Additionally, the results unveiled that the participants perceive value differently based on their perspective and the situation at hand. The SonarQube-provided severity score was the most frequently utilized parameter in the participants' valuation formulas, followed by file, cost, rule, and LOC. Moreover, participants were found to incorporate severity by mapping the severity level of a given TD item using one of three methods: linear, exponential, or relative sizing. The results also revealed that all but one participant fully relied upon and trusted the TD cost estimates provided by SonarQube. The one participant, instead, altered the provided estimates and conducted their prioritization based on the resulting alterations.

The findings above suggest that there is no silver bullet approach to TD prioritization that suits all developers' desires and needs, specifically in the context of the use of SonarQube when managing TD. However, though there is no 'one size fits all' approach to TD prioritization, it appears that balancing the trade off between the value and cost of TD is currently the most popular approach among this study's participants. TD prioritization is complex, challenging to perform, and variable depending on the environment in which it is conducted. Though SonarQube aids in expediting the TD prioritization process, developers were found to have significant differences in their respective TD prioritization approaches, even while utilizing the same static code analysis tool. Therefore, the software engineering community should continue supporting the different requirements and objectives of software developers when conducting TD prioritization through the means of developing more TD prioritization approaches, without concentrating on a specific prioritization perspective. Additionally, given that the value of a TD item can be subjective and context-dependent, one should look to incorporate multiple value estimation methods when developing a TD prioritization approach. Incorporating the ability to utilize multiple value estimation methods in an approach grants developers the ability to apply said approach in a greater number of circumstances and, potentially, have a greater impact.

CRediT authorship contribution statement

Reem Alfayez: Conceptualization, Data curation, Formal analysis, Project administration, Original draft, Writing – review & editing. **Robert Winn:** Conceptualization, Data curation, Formal analysis, Original draft, Writing – review & editing. **Wesam Alwehaibi:** Conceptualization, Data curation, Formal analysis, Original draft, Writing – review & editing. **Elaine Venson:** Conceptualization, Data curation, Formal analysis, Review & editing. **Barry Boehm:** Supervision, Review & editing.

Declaration of competing interest

No author associated with this paper has disclosed any potential or pertinent conflicts which may be perceived to have impending conflict with this work. For full disclosure statements refer to <https://doi.org/10.1016/j.infsof.2023.107147>.

Data availability

The data that has been used is confidential.

Acknowledgments

This material is based upon work supported in part by the U.S. Department of Defense through the Systems Engineering Research Center (SERC) under Contract H98230-08-D-0171. SERC is a federally funded University Affiliated Research Center managed by Stevens Institute of Technology.

References

- [1] P. Kruchten, R.L. Nord, I. Ozkaya, Technical debt: From metaphor to theory and practice, *IEEE Softw.* 29 (6) (2012) 18–21.
- [2] W. Cunningham, The WyCash portfolio management system, *SIGPLAN OOPS Mess.* 4 (2) (1992) 29–30, <http://dx.doi.org/10.1145/157710.157715>.
- [3] Z. Codabux, B. Williams, Managing technical debt: An industrial case study, in: 2013 4th International Workshop on Managing Technical Debt (MTD), 2013, pp. 8–15, <http://dx.doi.org/10.1109/MTD.2013.6608672>.
- [4] N. Rios, R.O. Spínola, M. Mendonça, C. Seaman, The practitioners' point of view on the concept of technical debt and its causes and consequences: A design for a global family of industrial surveys and its first results from Brazil, *Empir. Softw. Eng.* 25 (5) (2020) 3216–3287, <http://dx.doi.org/10.1007/s10664-020-09832-9>.
- [5] B. Curtis, J. Sappidi, A. Szykarski, Estimating the principal of an application's technical debt, *IEEE Softw.* 29 (6) (2012) 34–42, <http://dx.doi.org/10.1109/MS.2012.156>.
- [6] E. Tom, A. Aurum, R. Vidgen, An exploration of technical debt, *J. Syst. Softw.* 86 (6) (2013) 1498–1516, <http://dx.doi.org/10.1016/j.jss.2012.12.052>.
- [7] A. Martini, J. Bosch, M. Chaudron, Architecture technical debt: Understanding causes and a qualitative model, in: 2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications, 2014, pp. 85–92, <http://dx.doi.org/10.1109/SEAA.2014.65>.
- [8] R.L. Nord, I. Ozkaya, P. Kruchten, M. Gonzalez-Rojas, In search of a metric for managing architectural technical debt, in: 2012 Joint Working IEEE/IFIP Conference on Software Architecture and European Conference on Software Architecture, 2012, pp. 91–100, <http://dx.doi.org/10.1109/WICSA-ECSA.2012.17>.
- [9] S. Biffl, A. Aurum, B. Boehm, H. Erdogmus, P. Grünbacher, *Value-Based Software Engineering*, Springer Science & Business Media, 2006.
- [10] A. Ampatzoglou, A. Ampatzoglou, A. Chatzigeorgiou, P. Avgeriou, The financial aspect of managing technical debt, *Inf. Softw. Technol.* 64 (C) (2015) 52–73, <http://dx.doi.org/10.1016/j.infsof.2015.04.001>.
- [11] N. Zazworka, C. Seaman, F. Shull, Prioritizing design debt investment opportunities, in: Proceedings of the 2nd Workshop on Managing Technical Debt, MTD '11, Association for Computing Machinery, New York, NY, USA, 2011, pp. 39–42, <http://dx.doi.org/10.1145/1985362.1985372>.
- [12] M. M. Bomfim, V. A. Santos, Strategies for reducing technical debt in agile teams, in: T. Silva da Silva, B. Estácio, J. Kroll, R. Mantovani Fontana (Eds.), *Agile Methods*, Springer International Publishing, Cham, 2017, pp. 60–71.
- [13] E. Allman, Managing technical debt, *Commun. ACM* 55 (5) (2012) 50–55, <http://dx.doi.org/10.1145/2160718.2160733>.
- [14] D. Pina, A. Goldman, G. Tonin, Technical debt prioritization: Taxonomy, methods results, and practical characteristics, in: 2021 47th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2021, pp. 206–213, <http://dx.doi.org/10.1109/SEAA53835.2021.00034>.
- [15] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *J. Syst. Softw.* 101 (C) (2015) 193–220, <http://dx.doi.org/10.1016/j.jss.2014.12.027>.
- [16] R. Plösch, J. Bräuer, M. Saft, C. Körner, Design debt prioritization: A design best practice-based approach, in: 2018 IEEE/ACM International Conference on Technical Debt (TechDebt), 2018, pp. 95–104.
- [17] Y. Guo, R.O. Spínola, C. Seaman, Exploring the costs of technical debt management — a case study, *Empir. Softw. Engg.* 21 (1) (2016) 159–182, <http://dx.doi.org/10.1007/s10664-014-9351-7>.
- [18] R. Alfayez, W. Alwehaibi, R. Winn, E. Venson, B. Boehm, A systematic literature review of technical debt prioritization, in: Proceedings of the 3rd International Conference on Technical Debt, TechDebt '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 1–10, <http://dx.doi.org/10.1145/3387906.3388630>.
- [19] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, F.A. Fontana, A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools, *J. Syst. Softw.* 171 (2021) 110827.
- [20] P. Runeson, M. Host, A. Rainer, B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*, John Wiley & Sons, 2012.
- [21] SonarQube, 2020, <http://www.sonarqube.org>.
- [22] G.A. Campbell, P.P. Papapetrou, *SonarQube in Action*, first ed., Manning Publications Co., Greenwich, CT, USA, 2013.
- [23] P.C. Avgeriou, D. Taibi, A. Ampatzoglou, F. Arcelli Fontana, T. Besker, A. Chatzigeorgiou, V. Lenarduzzi, A. Martini, A. Moschou, I. Pigazzini, N. Saarimäki, D.D. Sas, S.S. de Toledo, A.A. Tsintzira, An overview and comparison of technical debt measurement tools, *IEEE Softw.* 38 (3) (2021) 61–71, <http://dx.doi.org/10.1109/MS.2020.3024958>.
- [24] E. Gama, M. Paixao, E.S.S. Freire, M.I. Cortés, Technical Debt's State of Practice on Stack Overflow: A Preliminary Study, Association for Computing Machinery, New York, NY, USA, 2019, <http://dx.doi.org/10.1145/3364641.3364668>.
- [25] C. Sterling, *Managing Software Debt: Building for Inevitable Change*, Addison-Wesley Professional, 2010.
- [26] K.S. Rubin, *Essential Scrum: A Practical Guide To the Most Popular Agile Process*, first ed., Addison-Wesley Professional, 2012.
- [27] J.-L. Letouzey, M. Ilkiewicz, Managing technical debt with the SQALE method, *IEEE Softw.* 29 (6) (2012) 44–51, <http://dx.doi.org/10.1109/MS.2012.129>.
- [28] V. Lenarduzzi, F. Lomio, H. Huttunen, D. Taibi, Are SonarQube rules inducing bugs? in: 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2020, pp. 501–511, <http://dx.doi.org/10.1109/SANER48275.2020.9054821>.
- [29] Sonarqube rules, 2020, <https://rules.sonarsource.com/>.
- [30] G. Digkas, M. Lungu, P. Avgeriou, A. Chatzigeorgiou, A. Ampatzoglou, How do developers fix issues and pay back technical debt in the Apache ecosystem? in: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER), 2018, pp. 153–163, <http://dx.doi.org/10.1109/SANER.2018.8330205>.
- [31] L.F. Ribeiro, M.A.D.F. Farias, M. Mendonça, R.O. Spínola, Decision Criteria for the Payment of Technical Debt in Software Projects: A Systematic Mapping Study, in: ICEIS 2016, SCITEPRESS - Science and Technology Publications, Lda, Setubal, PRT, 2016, pp. 572–579, <http://dx.doi.org/10.5220/0005914605720579>.
- [32] Y. Guo, C. Seaman, R. Gomes, A. Cavalcanti, G. Tonin, F.Q.B. Da Silva, A.L.M. Santos, C. Siebra, Tracking technical debt — An exploratory case study, in: 2011 27th IEEE International Conference on Software Maintenance (ICSM), 2011, pp. 528–531, <http://dx.doi.org/10.1109/ICSM.2011.6080824>.
- [33] N. Saarimäki, M.T. Baldassarre, V. Lenarduzzi, S. Romano, On the accuracy of SonarQube technical debt remediation time, in: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2019, pp. 317–324, <http://dx.doi.org/10.1109/SEAA.2019.00055>.
- [34] M.T. Baldassarre, V. Lenarduzzi, S. Romano, N. Saarimäki, On the diffuseness of technical debt items and accuracy of remediation time when using SonarQube, *Inf. Softw. Technol.* 128 (C) (2020) <http://dx.doi.org/10.1016/j.infsof.2020.106377>.
- [35] J. Tan, D. Feitosa, P. Avgeriou, An empirical study on self-fixed technical debt, in: Proceedings of the 3rd International Conference on Technical Debt, TechDebt '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 11–20, <http://dx.doi.org/10.1145/3387906.3388621>.
- [36] J. Tan, D. Feitosa, P. Avgeriou, Does it matter who pays back technical debt? An empirical study of self-fixed TD, *Inf. Softw. Technol.* 143 (C) (2022) <http://dx.doi.org/10.1016/j.infsof.2021.106738>.
- [37] V. Lenarduzzi, V. Mandić, A. Katin, D. Taibi, How long do junior developers take to remove technical debt items? in: Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), ESEM '20, Association for Computing Machinery, New York, NY, USA, 2020, <http://dx.doi.org/10.1145/3382494.3422169>.
- [38] A. Martini, F.A. Fontana, A. Biaggi, R. Roveda, Identifying and prioritizing architectural debt through architectural smells: A case study in a large software company, in: C.E. Cuesta, D. Garlan, J. Pérez (Eds.), *Software Architecture*, Springer International Publishing, Cham, 2018, pp. 320–335.

- [39] F.A. Fontana, I. Pigazzini, R. Roveda, D. Tamburri, M. Zanoni, E. Di Nitto, Arcan: A tool for architectural smells detection, in: 2017 IEEE International Conference on Software Architecture Workshops (ICSAW), 2017, pp. 282–285, <http://dx.doi.org/10.1109/ICSAW.2017.16>.
- [40] A. Martini, J. Bosch, Towards prioritizing architecture technical debt: Information needs of architects and product owners, in: 2015 41st Euromicro Conference on Software Engineering and Advanced Applications, 2015, pp. 422–429, <http://dx.doi.org/10.1109/SEAA.2015.78>.
- [41] CAST, 2020, www.castsoftware.com/products/health-dashboard.
- [42] T. Clancy, The standish group chaos report, in: Project Smart, 2014, pp. 8–9.
- [43] Kiuwan, 2020, <https://www.kiuwan.com>.
- [44] S.L. Jackson, Research Methods and Statistics: A Critical Thinking Approach, Cengage Learning, 2015.
- [45] X. Xia, L. Bao, D. Lo, Z. Xing, A.E. Hassan, S. Li, Measuring program comprehension: A large-scale field study with professionals, IEEE Trans. Softw. Eng. 44 (10) (2018) 951–976, <http://dx.doi.org/10.1109/TSE.2017.2734091>.
- [46] Ward explains debt metaphor, 2011, <http://wiki.c2.com/?WardExplainsDebtMetaphor>.
- [47] Technical debt, 2020, <https://martinfowler.com/bliki/TechnicalDebt.html>.
- [48] Sonarqube web API, 2019, <https://docs.sonarqube.org/display/DEV/Web+API>.
- [49] D.S. Cruzes, T. Dyba, Recommended steps for thematic synthesis in software engineering, in: 2011 International Symposium on Empirical Software Engineering and Measurement, 2011, pp. 275–284, <http://dx.doi.org/10.1109/ESEM.2011.36>.
- [50] J. He, Y. Zhou, A comparison of GAs using penalizing infeasible solutions and repairing infeasible solutions on restrictive capacity knapsack problem, in: Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO '07, Association for Computing Machinery, New York, NY, USA, 2007, p. 1518, <http://dx.doi.org/10.1145/1276958.1277247>.
- [51] S. McConnell, Software Estimation: Demystifying the Black Art, Microsoft Press, 2006.
- [52] K.C. Wong, A Domain-Dependent Approach to Determining File Importance, in: GCMS '13, Society for Modeling & Simulation International, Vista, CA, 2013.
- [53] P. Oman, J. Hagemester, Metrics for assessing a software system's maintainability, in: Proceedings Conference on Software Maintenance 1992, IEEE Computer Society, Los Alamitos, CA, USA, 1992, <http://dx.doi.org/10.1109/ICSM.1992.242525>, pp. 337, 338, 339, 340, 341, 342, 343, 344.
- [54] R.P. Buse, W.R. Weimer, A metric for software readability, in: Proceedings of the 2008 International Symposium on Software Testing and Analysis, ISTA '08, Association for Computing Machinery, New York, NY, USA, 2008, pp. 121–130, <http://dx.doi.org/10.1145/1390630.1390647>.
- [55] C. Chen, R. Alfayez, K. Srisopha, B. Boehm, L. Shi, Why is it important to measure maintainability and what are the best ways to do it? in: 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), 2017, pp. 377–378, <http://dx.doi.org/10.1109/ICSE-C.2017.75>.
- [56] M. Höst, B. Regnell, C. Wohlin, Using students as subjects—A comparative study OfStudents and professionals in lead-time impact assessment, Empir. Softw. Engg. 5 (3) (2000) 201–214, <http://dx.doi.org/10.1023/A:1026586415054>.
- [57] J. Carver, L. Jaccheri, S. Morasca, F. Shull, Issues in using students in empirical studies in software engineering education, in: Proceedings. 5th International Workshop on Enterprise Networking and Computing in Healthcare Industry (IEEE Cat. No.03EX717), 2003, pp. 239–249, <http://dx.doi.org/10.1109/METRIC.2003.1232471>.
- [58] P. Ralph, N.b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, et al., Empirical standards for software engineering research, 2020, arXiv preprint [arXiv:2010.03525](https://arxiv.org/abs/2010.03525).